

Reinforcement Learning

Deep Reinforcement Learning

Based on An Introduction to Deep
Reinforcement Learning**

+ Sutton/Barto* Sections 9.7 and 15.7

Michael Hahsler, SMU

*Sutton and Barto, Reinforcement Learning: An Introduction, 2nd edition,
MIT Press, Cambridge, MA, 2018

**Vincent François-Lavet, Peter Henderson, Riashat Islam, Marc G. Bellemare
and Joelle Pineau (2018), "An Introduction to Deep Reinforcement Learning",
Foundations and Trends in Machine Learning: Vol. 11, No. 3-4. DOI:
10.1561/22000000071.



Online Material



Topics of this Course

- Introduction to reinforcement learning
- Markov decision processes
- Part I: Tabular Methods
 - Dynamic programming
 - Monte Carlo methods
 - Temporal-difference learning
 - Multi-step bootstrapping
 - Planning and learning with tabular methods
- Part II: Approximate Solution Methods
 - Prediction and Control using Approximation
 - Eligibility Traces
 - Policy Gradient Methods
- **Part III: Modern RL Methods**
 - **Deep Reinforcement Learning**
 - Current Applications

Summary of Notation*

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\mathbf{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\mathbf{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bold)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X , i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\operatorname{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
Q, Q_t	array estimates of action-value function q_π or q_*

Temporal Difference Learning

U_t	target for estimate at time t
δ_t	TD error

*Some authors use a slightly different notation.

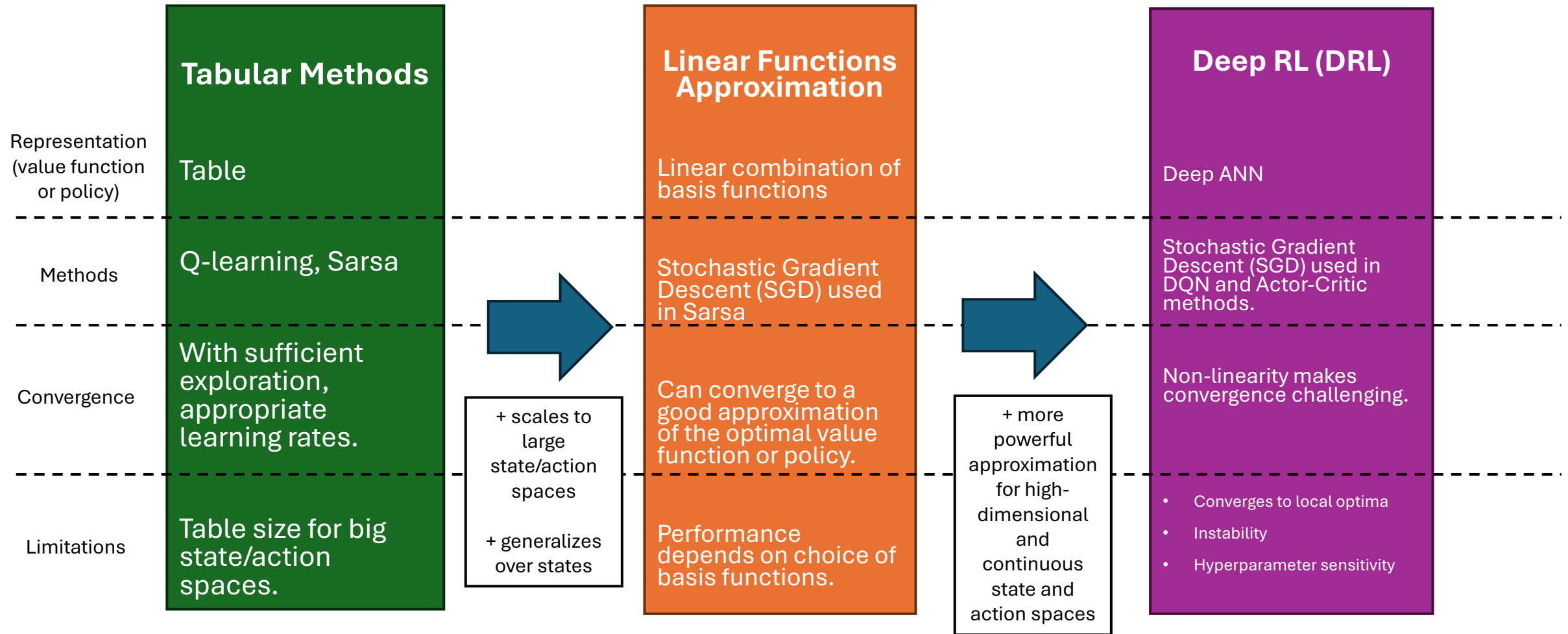
MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r s, a)$	probability of transition to state s' and receiving reward r , from state s taking action a .
$p(s' s, a)$	probability of transition to state s' from state s taking action a .
$r(s, a)$	expected immediate reward from state s after action a .
$r(s, a, s')$	expected immediate reward from state s to s' with action a .
$\pi(a s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

Approximation

$\mu(s)$	on policy distribution (fraction of time spent in state)
α	step size (gradient descent learning rate)
$\hat{v}(s, \mathbf{w})$	approximate value function
$\hat{q}(s, a, \mathbf{w})$	approximate action-value function
$\pi(a s, \boldsymbol{\theta})$	approximate policy
$\widehat{VE}$	mean squared value error
$\nabla f(x)$	gradient of function f evaluated at x

From Tabular RL to Deep RL



Disclaimer: Convergence

- So far, we have had strong convergence guarantees:
 - **Tabular RL** has the contraction mapping guarantee (Bellman Operator): guaranteed convergence to the optimal value function (a fixed point). The policy improvement theorem shows that the policy improves in every step.
 - **TD/Q-learning** converges with sufficient exploration and diminishing learning rates.
 - **Linear approximation**: Mean squared value error creates a convex optimization problem. On-policy learning converges to the global optimum. But off-Policy convergence is not guaranteed: Off-policy + bootstrapping + function approximation = **deadly triad**
- Non-linear approximation (DRL) breaks most guarantees in RL!
 - **Loss of contraction mapping** due to a non-linear projection. A fixed point may not exist or be reachable.
 - **Monotonic policy improvement is lost**: Policy can get worse.
 - **Stochastic approximation guarantees break**: Targets depend on parameters (bootstrapping), and **gradients are biased**.

Value-based DRL Methods

Use ANNs to represent a parameterized value function that is updated via gradient descent.

Remember: Value Functions

- Immediate reward: $r_t = r(s_t, a_t, s_{t+1})$
- Value functions:
 - $V^\pi(s) = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid S_t = s, \pi]$
 - $Q^\pi(s, a) = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid S_t = s, A_t = a, \pi]$

The expectation reflects all uncertainty:

- Stochastic policy
- Stochastic transitions
- Stochastic reward model

- Bellman equation:

$$Q^\pi(s, a) = \sum_{s' \in \mathcal{S}} p(s, a, s') \left(r(s, a, s') + \gamma Q^\pi(s', a = \pi(s')) \right)$$

- Optimality condition:

$$V^*(s) = \max_{\pi \in \Pi} V^\pi(s) \quad \text{and} \quad Q^*(s, a) = \max_{\pi \in \Pi} Q^\pi(s, a)$$

- Optimal policy: $\pi^*(s) = \operatorname{argmax}_{a \in \mathcal{A}} Q^*(s, a)$

New: Advantage Function

- The “advantage” of choosing action a over the baseline given by the state value is

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

- Positive advantage means action a is better than $\pi(s)$
For the optimal π^* :

$$A^{\pi^*}(s, a) = \begin{cases} 0 & \text{if } s = s^* \\ < 0 & \text{otherwise.} \end{cases}$$

- V , Q and A can be estimated using Monte Carlo methods (unbiased).
- V , Q and A can be used to calculate a TD-error.

Remember: Tabular Q-Learning

- An extremely influential method developed by Watkins (1989).
- Tabular update rule only uses Sars:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[\overbrace{R_{t+1}}^{\text{target}} + \gamma \max_a \underbrace{Q(S_{t+1}, a)}_{\text{prediction}} - \underbrace{Q(S_t, A_t)}_{\text{TD-error}} \right]$$

- Q directly approximates q_* independent of the behavior policy being followed!
→ Off-policy
- Converges to the optimal value function if
 - State-action pairs are represented discretely (in a table).
 - All action are repeatedly sampled in all states (behavior policy keeps exploring).
- **Issue:** $Q(s, a)$ can often not be represented in a table due to too many action-value pairs. Use a parameterized approximation $Q(s, a; \theta)$. Deadly triade:
 - Bootstrapping
 - off-policy learning
 - function approximation
- Linear approximation converges for special cases. Non-linear approximation is an issue!

Fitted Q-Learning

Notation: DRL literature uses often Q where our textbook uses $\hat{q}$

- Use a parameterized approximation $Q(s, a; \theta)$
- Initial setting for θ_0 such that all Q-value are close to 0 (helps with learning).

- Target value at iteration k :

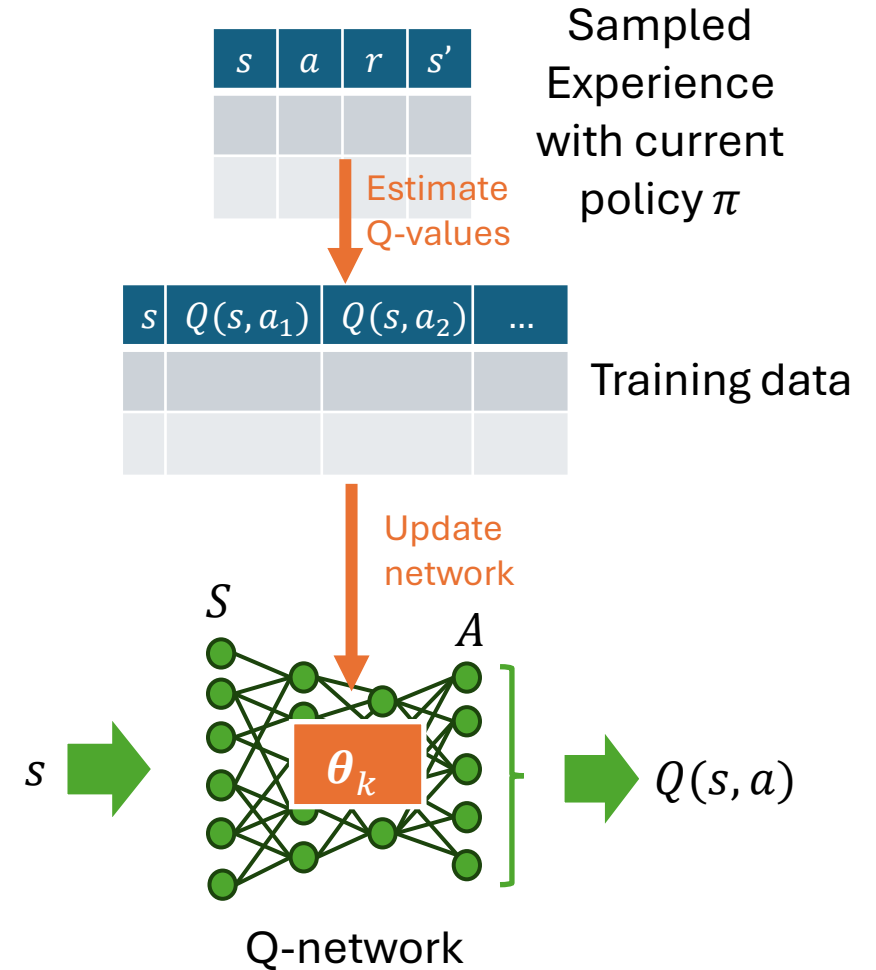
$$U_k = r + \gamma \max_{a'} Q(s', a'; \theta_k)$$

- Update:

$$\theta_{k+1} = \theta_k + \alpha (U_k - Q(s, a, \theta_k)) \nabla Q(s, a, \theta_k)$$

Neural fitted Q-learning (NFQ; Riedmiller, 2005)

- Use an ANN called the **Q-network** as the approximator for the Q-function.
- **Issue:** Online learning does not work well with our hardware. (GPUs efficiently perform many matrix operations in parallel)
- **Solution:**
 - Learn the Q-network offline: **batch learning** with **experience replay** from a sample set of experience containing (s, a, r, s') tuples.
 - Use **supervised learning (regression)** with a sample from a replay buffer as the training set.
 - The regression target (Q-values) are estimated from the tuples using **temporal differencing**: $Q(s, a) = r + \gamma \max_{a'} Q(s', a')$
- **More issues:**
 - May not converge or become unstable (uses approximation).
 - Q-values tend to be overestimated due to the max operator in the target.

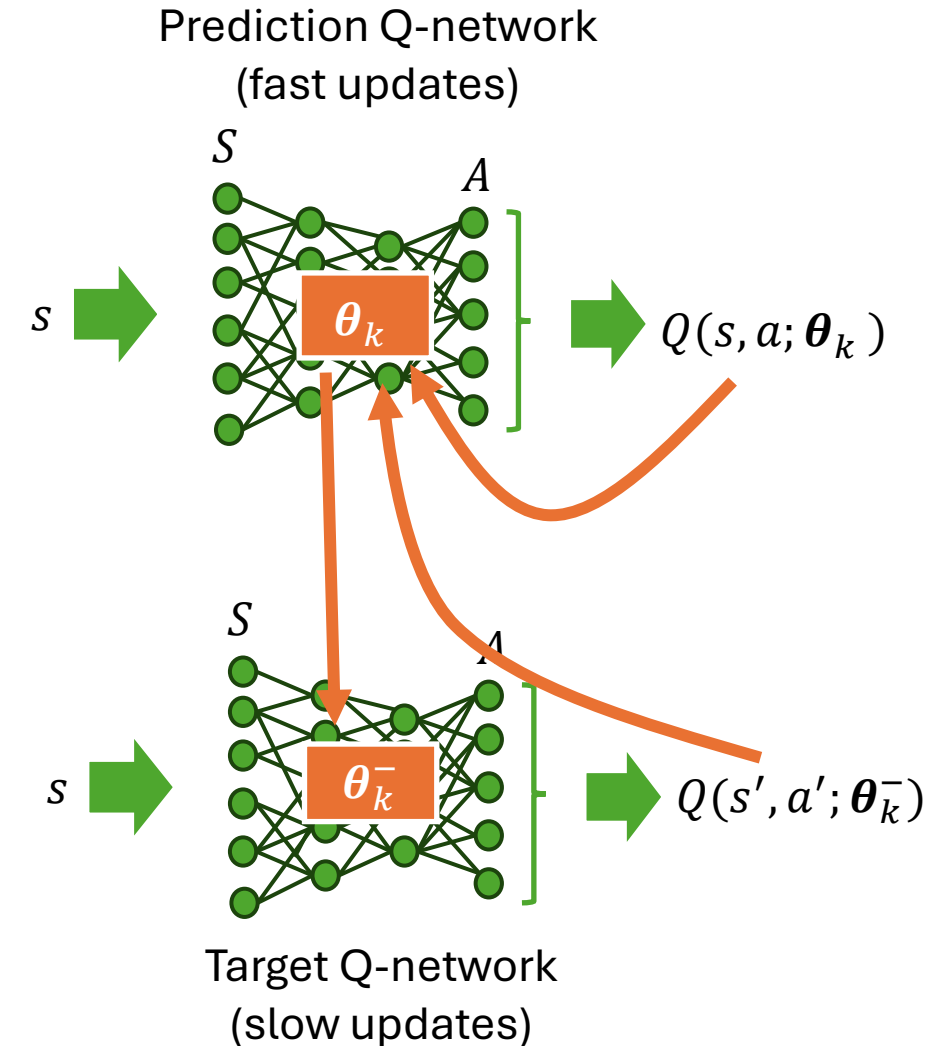


Deep Q-Network (DQN; Minh et al, 2015)

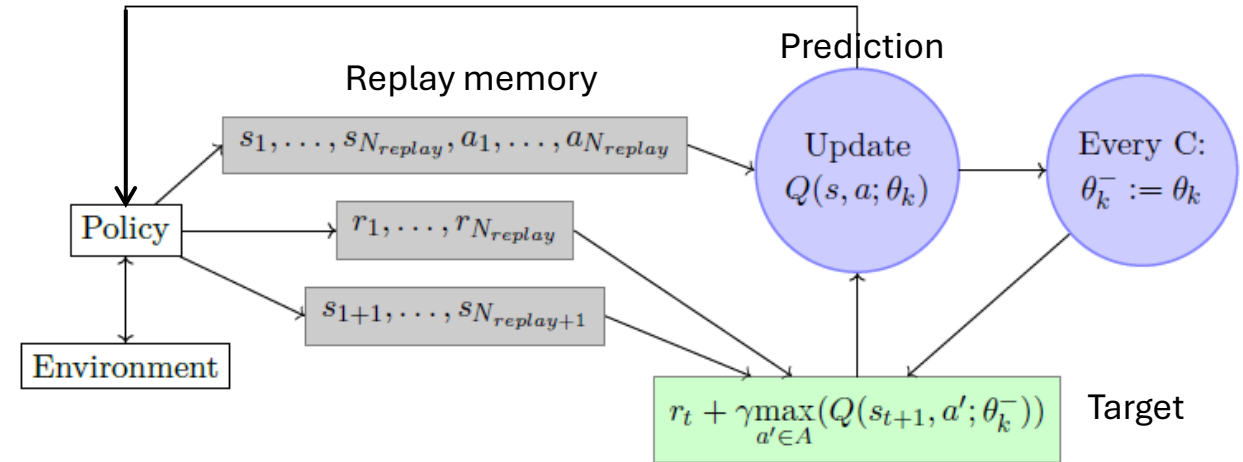
- DQN addresses the **instability** of NFQ.
- **Idea:** reduce instability by calculating the target from a second Q-network that is updated slowly:

$$U_k = r + \gamma \max_{a'} Q(s', a'; \theta_k^-)$$

- **Updates:**
 - θ_k is updated every iteration:
 $\theta_{k+1} = \theta_k + \alpha (U_k - Q(s, a, \theta_k)) \nabla Q(s, a, \theta_k)$
 - θ_k^- is updated only every C iterations by $\theta_k^- = \theta_k$.
- **Effect:** Target U_k estimates are kept constant for C iterations, instabilities cannot propagate quickly.



DQN in Online Settings



1. Initialize θ_0 and θ_0^- so that all $Q(s, a; \theta_0)$ are close to 0.
2. Use an ϵ -greedy policy to collect experience in the form of N_{replay} many (s, a, r, s') tuples.
3. Mini-batches (e.g., 32 elements) are taken randomly from the replay memory to update θ_k .
4. θ_k^- is only set to θ_k every C iterations.
5. Go back to collecting more experience with a new policy ϵ -greedy policy extracted from the prediction network.

Double DQN (DDQN; Van Hasselt, 2016)

- **Issue:** Q-learning and DQN use one function in the target to choose the action and evaluate its value. The max creates an **overestimation bias**.

$$\theta_{k+1} = \theta_k + \alpha \underbrace{\left(\overbrace{r + \gamma \max_{a'} Q(s', a'; \theta_k)}^{\text{target}} - \underbrace{Q(s, a, \theta_k)}^{\text{prediction}} \right)}_{\text{TD-error}} \nabla Q(s, a, \theta_k)$$

- DDQN **decoupling the action selection and evaluation** by using different parameters:

$$U_k = r + \gamma Q(s', \underset{a}{\operatorname{argmax}} Q(s', a'; \theta_k); \theta_k^-)$$

Action evaluation uses
more stable θ_k^-

Action selection
uses θ_k

Dueling Network Architecture (Wang, 2015)

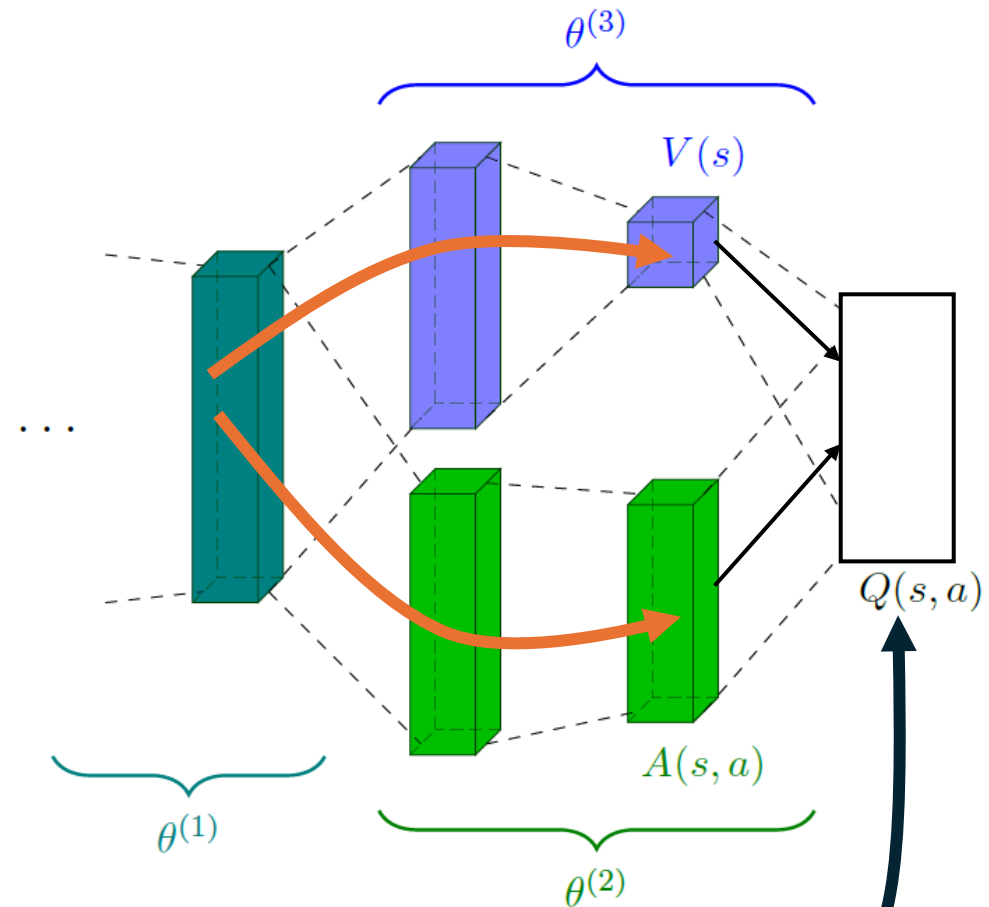
- Dueling Network Architecture only changes the ANN architecture. It learns two components:
 - Value function $V(s; \theta^{(1)}, \theta^{(3)})$
 - Advantage function $A(s, a'; \theta^{(1)}, \theta^{(3)})$
- $Q(s, a)$ is calculated using the equation below.

- Reminder: Advantage function

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

gives the “advantage” of choosing action a over the state value.

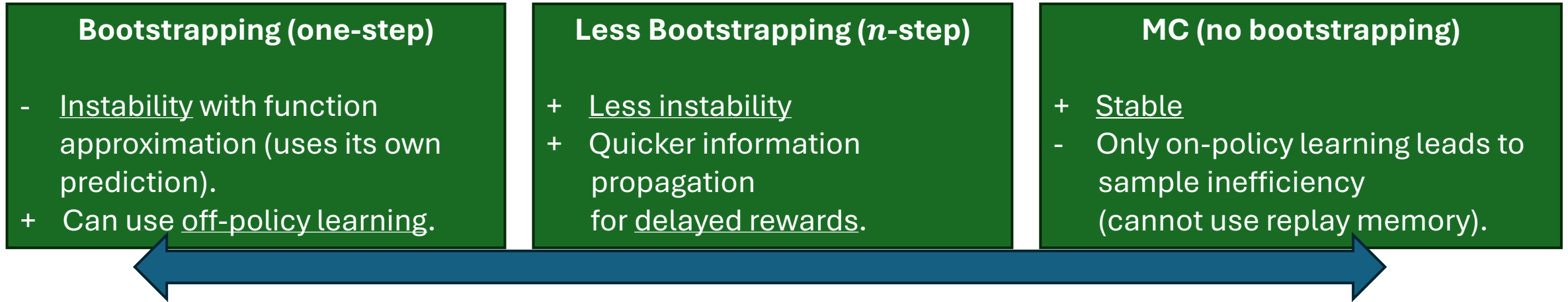
- This approach typically leads to improved performance.



$$Q(s, a; \theta^{(1)}, \theta^{(2)}, \theta^{(3)}) = V(s; \theta^{(1)}, \theta^{(3)}) + \left(A(s, a; \theta^{(1)}, \theta^{(2)}) - \max_{a' \in \mathcal{A}} A(s, a'; \theta^{(1)}, \theta^{(2)}) \right)$$

Becomes 0 for $a = a^*$

n -step Methods



- **n -step** methods are between Q-learning (one-step) and MC, which does not bootstrap.
- Use n -step target for DQN-type algorithms:

$$U_k^n = \sum_{t=0}^{n-1} \gamma^t r_t + \gamma^n \max_{a' \in \mathcal{A}} Q(s_n, a'; \boldsymbol{\theta}_k)$$

- **Traces** can also be used.

Policy Gradient DRL Methods

Use ANNs to represent and learn a parameterized policy.

Remember: Policy Gradient Theorem

Notation: θ and w are often used interchangeably!

- We define the performance measure as the true value of following π_θ from the start state s_0

$$J(\theta) \stackrel{\text{def}}{=} v_{\pi_\theta}(s_0) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s_0 \right]$$

- The **policy gradient theorem** gives us an analytical expression for the unbiased gradient of the performance with respect to the policy parameters:

$$\nabla J(\theta) = \mathbb{E}_{\pi} [q_{\pi}(s, a) \nabla \log \pi(a|s)] \propto \sum_s \mu(s) \sum_a q_{\pi}(s, a) \nabla \pi(a|s, \theta)$$

On-policy
distribution under π

Depends on
estimates for q

Depends on the policy
approximation function.

- **Convergence guarantee:**

- Gradient is unbiased and with reducing step size appropriately, ascent **will converge to a local maximum** of $J(\theta)$.
- **Does NOT guarantee: global optimum, fast convergence or stability (with non-linear approximation)**

Notation: θ and w are often used interchangeably!

Policy Gradient

- Following the ideas of REINFORCE.
- Gradient of a stochastic policy

Needs a value function estimate

$$\nabla_w v_{\pi_w}(s_0) = \mathbb{E}_{s \sim \mu_{\pi_w}, a \sim \pi_w} [\nabla_w (\log \pi_w(s, a)) Q^{\pi_w}(s, a)]$$

Sample state from the on-policy distribution and action from the policy

- Gradient of a differentiable deterministic policy

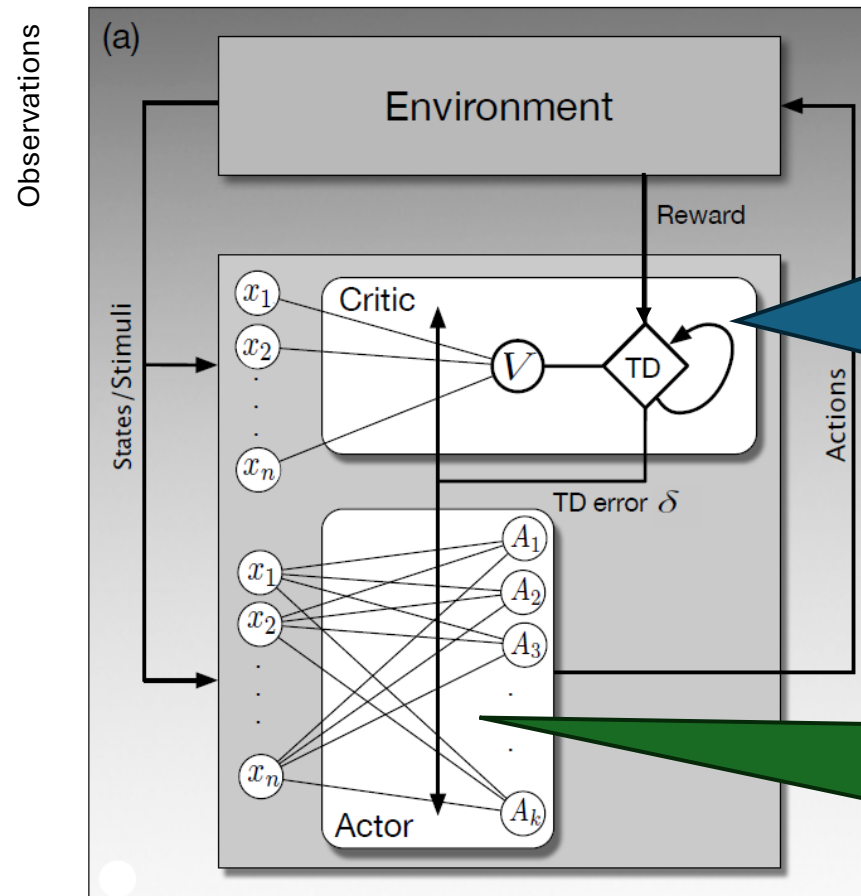
$$\nabla_w v_{\pi_w}(s_0) = \mathbb{E}_{s \sim \mu_{\pi_w}} [\nabla_w (\pi_w) \nabla_a (Q^{\pi_w}(s, a)) | a = \pi_w(s)]$$

Also requires this gradient!

Notation: θ and w are often used interchangeably!

Neural Actor-Critic Method

Remember: Estimate the value function and a parameterized policy at the same time.



Critic

- Represents value function $Q^{\pi_w}(s, a) \approx Q(s, a; \theta)$
- Produces the TD error from the reward signal using its network's state-value prediction.
- Adjusts state-value network using the TD error.
- Does not select actions!

Actor

- Represents policy π_w
- Selects actions with a policy network.
- Adjusts a policy network based on the TD error
- No access to reward signal!

Section 15.7 in Sutton and Barto (2018)

Neural Actor-Critic Method: One-Step Algorithm

One-step Actor–Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

actor

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

critic

Parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

Initialize S (first state of episode)

$I \leftarrow 1$

Loop while S is not terminal (for each time step):

$A \sim \pi(\cdot|S, \theta)$

Take action A , observe S', R

$\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$

TD error: how much is G better than $\hat{v}$

(if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$

Update the critic (value function) = TD(0)

$\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$

Update the actor (policy) = REINFORCE

$I \leftarrow \gamma I$

$S \leftarrow S'$

Takes care of
discounting γ^t

Possible improvements:

Sample efficiency: Use a replay memory with a set of (s, a, r, s') tuples.

Stability and faster reward propagation: Multi-step methods

Notation: θ and w are often used interchangeably!

Natural Policy Gradients (NPG)

- Regular policy gradient methods
 - Update policy parameters $\mathbf{w}$ in the direction of the steepest ascent of the performance measure $J(\mathbf{w}) \stackrel{\text{def}}{=} v_{\pi_{\mathbf{w}}}(s_0)$
 - The update $\Delta \mathbf{w} \propto \nabla_{\mathbf{w}} J(\mathbf{w})$ maximizes $J(\mathbf{w}) - J(\mathbf{w} + \Delta \mathbf{w})$. Using the step size α creates a constraint on $\|\Delta \mathbf{w}\|_2$.
 - **Issue:** The same amount of change in the parameter space $\mathbf{w}$ can lead to very different magnitudes of change in the objective function vs. the actual policy $\pi_{\mathbf{w}}$ (the probability distribution over actions). This results in **instability**.
- **Idea:** Natural gradients
 - Constraint $\Delta \mathbf{w}$ by how much it changes the policy.
 - The Kullback-Leibler (KL) divergence measures the difference between two policies' action probability distributions $D_{KL}(\pi_{\mathbf{w}} || \pi_{\mathbf{w} + \Delta \mathbf{w}})$ and we could constraint $\Delta \mathbf{w}$ by a maximum allowable change in D_{KL} .
 - Local approximation: The Fisher information criterion can be used to **adjust the gradient** for changes in policy distribution. α is now a constraint on policy changes:

$$\Delta \mathbf{w} \propto F_{\mathbf{w}}^{-1} \nabla_{\mathbf{w}} J(\mathbf{w})$$

- **Problem:** Calculating the Fisher information matrix ($F_{\mathbf{w}}$) is too expensive for deep learning models. We need an approximation for the approximation!

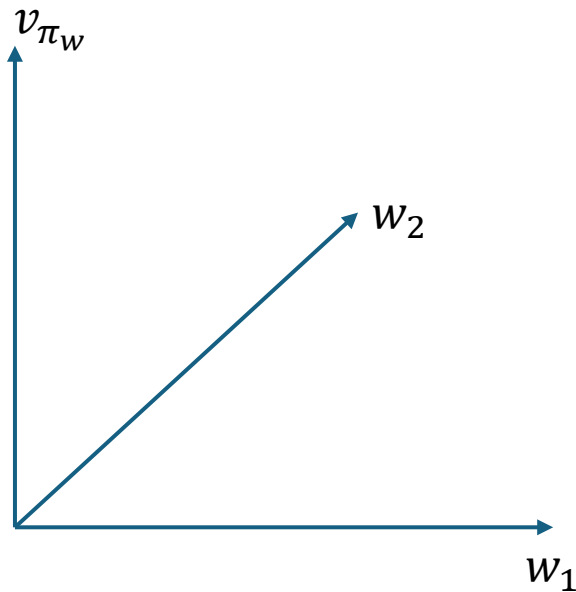
The idea inspired several state-of-the-art methods:

- Trust Region Optimization (TRPO)
- Proximal Policy optimization (PPO)

Notation: θ and w are often used interchangeably!

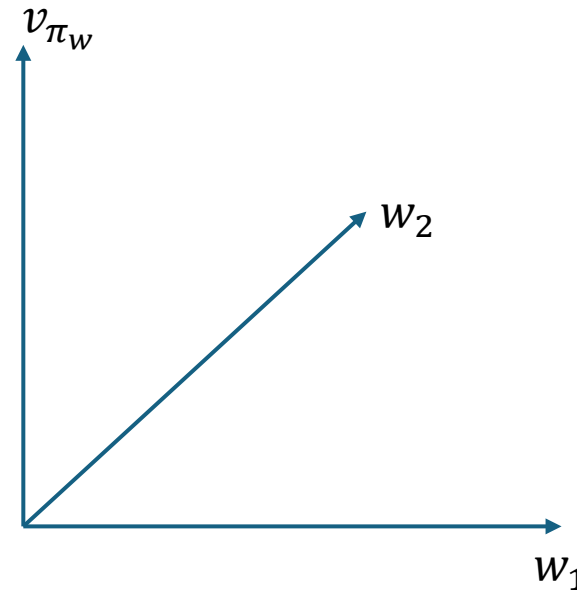
Standard vs. Natural Policy Gradient

Standard Gradient

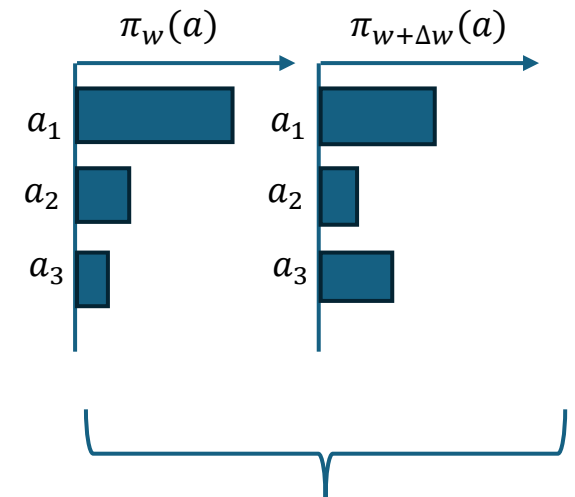


Move a fixed distance in the **parameter space**. This can lead to large policy changes.

Natural Policy Gradient



Move a fixed distance in the **policy space**.



$D_{KL}(\pi_w || \pi_{w+\Delta w})$

Trust Region Policy Optimization (TRPO; Schulman et al, 2015)

Notation: θ and w are often used interchangeably!

- Simplification of NPG.
- Finds the update that improves the advantage function the most

$$\max_{\Delta w} \mathbb{E}_{s \sim \mu^{\pi_w}, a \sim \pi_w} \left[\frac{\pi_{w+\Delta w}(s, a)}{\pi_w(s, a)} A^{\pi_w}(s, a) \right]$$

Prefer new policies that pick high-advantage actions.

- constrained by an explicitly bound the policy change per update step.

$$\mathbb{E} [D_{KL}(\pi_w(s, \cdot) || \pi_{w+\Delta w}(s, \cdot))] \leq \delta \quad \text{where } \delta \in \mathbb{R}$$

- δ is a tuneable hyperparameter specifying the size of the “trust region.”

Proximal Policy Optimization (PPO; Schulman et al, 2017)

Notation: θ and w are often used interchangeably!

- A popular and more practical variant of TRPO.
- Instead of KL-divergence, it uses a clipped surrogate objective function to penalize overly large policy changes with at probability ratio outside of $[1 - \epsilon, 1 + \epsilon]$

$$r_t(w) = \frac{\pi_{w+\Delta w}(s, a)}{\pi_w(s, a)}$$

$$\max_{\Delta w} \mathbb{E}_{s \sim \mu^{\pi_w}, a \sim \pi_w} \left[\min(r_t(w) A^{\pi_w}(s, a), \text{clip}(r_t(w), 1 - \epsilon, 1 + \epsilon) A^{\pi_w}(s, a)) \right]$$

- ϵ is a tunable hyperparameter
- Achieves much of the stability of NPG methods with the computational efficiency of first-order optimization.

Important Trick: Experience Replay

- Experience replay uses memory to store interactions (s, a, r, s') and reuses them for learning again later.
- This addresses:
 - Sample efficiency issues
 - Neural network's problems with forgetting
 - Creates uncorrelated training data because episodes are broken.
 - Enables the use of efficient NN learning methods like mini-batch SGD.
- It requires an off-policy learning algorithm.

What you Should Know

- DRL uses neural networks for non-linear approximation.
- This is
 - more powerful for high-dimensional and continuous state and action spaces, and
 - does not need manual state feature creation.
- however
 - most convergence guarantees are gone (for semi-gradient methods, policy gradient theorem).
- Instability from the “Deadly Triad” (e.g., in DQN)
 - Non-linear function approximation (neural networks)
 - Bootstrapping (updating targets using current estimates)
 - Off-policy learning (learning from data generated by a different policy)
- Lead to
 - Small errors get amplified through bootstrapping
 - The network chases a moving target
 - Training can diverge or oscillate wildly
- Many tricks are needed
 - Target networks → reduce moving targets
 - Experience replay → decorrelate data
 - Double Q-learning → reduce overestimation
 - Advantage normalization → reduce variance
 - Entropy regularization → improve exploration

